DMQ: Direct Mixed-Precision Quantization for Fixed-Memory-Constrained GPU LLM Deployment

Zhaoning Zhang[1] , Zhongxing Li[1]

[1] National Key Laboratory of Parallel and Distributed ComputingCollege of Computer Science and TechnologyNational University of Defense Technology, Changsha, China

Abstract. Large language models (LLMs) are increasingly being deployed on resource-constrained edge devices and consumer-grade GPUs, where stringent memory budgets often impede the adoption of high-accuracy models. Mixed-precision quantization emerges as a promising solution to this dilemma, yet existing methods either rely on time-consuming search algorithms or demand complex fine-tuning processes. In this paper, we propose DMQ (Direct Mixed-Precision Quantization), a fast and flexible framework tailored explicitly for fixed-memory deployment scenarios. Distinct from prior works that depend on extensive profiling or local reconstruction loss, DMQ calculates operator-wise quantization error by simulating quantization to target bit-widths and computing loss in a parameter update-free manner, thus efficiently capturing the intrinsic error induced by quantization. Furthermore, we design a rule-based sensitivity adjustment mechanism that accounts for three key factors: Hessian diagonal correction, down-projection compression ratio, and Softmax numerical distribution of v-projection layers, which effectively preserves the models critical components from aggressive low-bit quantization. We also formulate the precision allocation problem as a classic grouped knapsack problem and solve it to global optimality via dynamic programming. Extensive experiments on mainstream LLMs demonstrate that DMQ achieves state-of-the-art accuracy with negligible search overhead (milliseconds-level), significantly reducing the models memory footprint while maintaining superior performance across diverse model architectures and deployment constraints. Source code is available at https://github.com/zzningxp/DartMQ.

Keywords: Mixed-precision quantization, Fixed-Memory Deployment, Hessian Diagonal Correction, Dynamic Programming.

1 INTRODUCTION

Large Language Models (LLMs) have revolutionized natural language processing, yet their deployment faces severe bottlenecks in resource-constrained embedded and edge systems. Unlike cloud environments with elastic memory, embedded devices operate under fixed, physically rigid memory budgets. In these scenarios, exceeding the memory limit even by a single byte results in catastrophic Out-Of-Memory (OOM) failures, rendering the system unusable.

Quantization has emerged as the primary technique to compress LLM weights from 16-bit floating-point to low-bit integer formats. However, existing fixed-precision methods (e.g., GPTQ[1], AWQ[2]) suffer from a fundamental inflexibility: uniformly quantizing to 4-bit (INT4) may fit the model but causes unacceptable accuracy degradation for complex reasoning, while 8-bit (INT8) preserves accuracy but often exceeds the strict memory limits of embedded hardware. This "all-or-nothing" trade-off makes fixed-precision approaches ill-suited for the diverse and tight memory constraints typical of embedded deployments.

The core limitation of uniform quantization is the Memory-Accuracy Staircase effect. Due to the discrete nature of bit-widths, uniform methods can only operate at fixed memory levels. For instance, quantizing an 8B-parameter model uniformly to 4-bit requires approximately 4.5 GB of memory (including overhead), while dropping to 3-bit reduces this to roughly 3.4 GB. This creates a 1.1 GB "dead zone" of unused memory capacity for any device with VRAM between these two thresholds. In such cases, uniform quantization forces a binary choice: either fit the model with significant accuracy loss (3-bit) or fail to load it entirely (4-bit). However, this memory gap represents sufficient capacity to upgrade critical layers to higher precision, recovering substantial perplexity drops without exceeding the hardware limit, which uniform methods inherently cannot exploit. This necessitates a mixed-precision approach that assigns varying bit-widths to different model components, to fill the memory gap precisely and maximize accuracy within the exact hardware constraints.

Meanwhile, existing mixed-precision methods face two core challenges: first, most rely on heuristic search strategies, which bring prohibitive computational overhead and cannot guarantee global optimality under fixed memory constraints; second, they commonly use local reconstruction loss as a proxy for global model degradation, while our empirical analysis reveals a significant discrepancy between this local loss and the actual impact on global perplexity (PPL), leading to misallocation of bit-widths for critical layers.

In this work, we propose DMQ (Direct Mixed-Precision Quantization), a deterministic, low-overhead framework tailored for fixed-memory-constrained LLM deployment on embedded systems. Unlike previous works that depend on extensive profiling or reconstruction loss, DMQ calculates operator-wise quantization loss by performing forward passes on the FP16 model, simulating quantization to target bit-widths, and computing the GPTQ loss without updating parameters. This efficient strategy captures the intrinsic error introduced by quantization. Furthermore, we introduce a rule-based sensitivity adjustment that accounts for the compression ratio (e.g., in down-projection operator) and the numerical distribution (specifically for V-projection layers), ensuring critical components are preserved. Our core insight is transforming the precision allocation problem into a classic Grouped Knapsack Problem, solvable to global optimality via Dynamic Programming (DP). Unlike heuristic searches, our DP solver provides a deterministic execution path with millisecond-level latency, ensuring that the same memory budget always yields the identical, optimal configuration.

Our main contributions are summarized as follows:

1. We propose a novel mixed-precision framework specifically designed for hard memory constraints in embedded systems, prioritizing “fit-first, optimize-second” to guarantee deployability.
2. We formulate precision allocation as a grouped knapsack problem and solve it via deterministic dynamic programming, ensuring global optimality within the discrete candidate set and real-time configurability.
3. We introduce a directly operator-wise sensitivity weighting mechanism calibrated via one-time offline profiling, bridging the gap between local quantization error and global model perplexity with negligible deployment overhead.
4. Extensive experiments demonstrate that DMQ outperforms state-of-the-art fixed-precision quantization methods in accuracy under identical memory budgets, while reducing search time from hours to minutes, enabling efficient and reliable LLM deployment on embedded platforms.

2 RELATED WORK

2.1 Weight Offloading for LLM Inference

Weight offloading enables large model inference on memory-limited GPUs by dynamically swapping weights between VRAM and CPU memory [3], with optimizations like page-based memory management in vLLM. However, it suffers from unpredictable PCIe transfer latency, high-concurrency throughput degradation, and no actual model size reduction.

2.2 Post-Training Quantization for LLMs

Post-training quantization (PTQ) is the mainstream LLM compression method without full fine-tuning. Representative fixed-precision PTQ works include GPTQ [1] (second-order optimization for accurate 4-bit quantization), AWQ [2] (activation-aware weight scaling), and SmoothQuant [4] (activation distribution smoothing for INT8 quantization). All these methods are limited to fixed bit-widths, which cannot flexibly adapt to tight, diverse memory budgets, leading to an unavoidable trade-off between model fit and accuracy.

Mixed-Precision PTQ for LLMs Existing mixed-precision PTQ methods address fixed-precision inflexibility, but have critical limitations: HAWQv3 [5] requires time-consuming Hessian calculation and layer-wise fine-tuning; PTQ4Bit [6] relies on heuristic rules that cannot guarantee global optimality under fixed memory constraints; AMQ [7] uses evolutionary search with non-deterministic results and proxy model approximation errors. Quantization-aware training (QAT) methods like MQAT [8] support mixed precision, but require prohibitive full-model retraining for billion-scale LLMs.

2.3 Combinatorial Optimization for Resource Allocation

Mixed-precision precision allocation is essentially a combinatorial optimization problem. Traditional solutions like greedy search or reinforcement learning either converge to local optima or require heavy training. While the classic grouped knapsack problem (GKP) has been applied to channel pruning [9] and tensor decomposition.

3 METHODOLOGY

3.1 Problem Formulation

We target the deployment of transformer-based LLMs on consumer-grade GPUs with a fixed VRAM budget. We take each linear matrix multiplication operator in the transformer as the basic allocation unit.

Formally, we define a model as a set of N operators $\mathcal{O} = \{o_1, o_2, \ldots, o_N\}$. For each operator o_i, we have a fixed set of candidate precision levels $\mathcal{B} = \{3,4,5,6,8\}$ (in bits). For each operator o_i and precision $b \in \mathcal{B}$, we precompute two core attributes:

- $v_{i,b}$: The VRAM footprint of operator o_i when quantized to precision b, calculated based on the number of parameters, bit-width, group size (fixed to 128), and the additional overhead of zero points (+0.25 bits per weight, bpw).
- $e_{i,b}$: The per-channel reconstruction error of operator o_i when quantized to precision b using GPTQ, corrected by sensitivity, which measures the quantization noise of the operator.

For fixed-max-length context scenarios, we pre-allocate a fixed KV Cache budget from the total GPU VRAM, resulting in a fixed maximum VRAM budget V_{max} for model parameters. Our optimization goal is to assign a precision $b_i \in \mathcal{B}$ to each operator o_i, such that the total VRAM footprint of the model does not exceed V_{max}, and the overall quantization error (calibrated by operator sensitivity) is minimized. The problem is formalized as:

$$\min_{\{b_i\}} \sum_{i=1}^{N} s_i \cdot e_{i,b_i} \tag{1}$$

$$s.t. \sum_{i=1}^{N} v_{i,b_i} \leq V_{max} \tag{2}$$

where s_i is the quantization sensitivity of operator o_i, which calibrates the contribution of the operator's quantization error to the end-to-end model PPL.

3.2 Key Insights from Empirical Analysis

Staircase Effect and Error Compensation We observe a pronounced staircase effect in local quantization error: increasing the bit-width by a single unit often reduces the local reconstruction error by an order of magnitude. However, this drastic

local error reduction only brings marginal improvement in global PPL, as quantization errors in LLMs exhibit complementary robustness across layers, where errors in one layer can be partially absorbed by subsequent layers. This exposes the weakness of greedy strategies, which tend to over-invest memory in layers with large local error drops but negligible global utility. Our grouped knapsack formulation avoids this trap by evaluating the global sensitivity-weighted gain per unit memory for all discrete transitions simultaneously.

Smooth Logarithmic Pareto Frontier Extensive empirical analysis across mainstream LLM architectures reveals that the memory-accuracy Pareto frontier forms a smooth, continuous, logarithmic-like curve. This smoothness arises from three intrinsic properties: the additivity of quantization loss across layers, the monotonicity between memory savings and bit-width reduction, and the uniform sensitivity distribution of operators in dense Transformer models. This implies that exhaustive heuristic searches of the entire frontier are computationally wasteful; for fixed memory constraints, we only need to pinpoint the single optimal configuration that satisfies the hard budget, motivating our direct solving approach.

Sensitivity Discrepancy Between Local Loss and Global PPL A fundamental assumption in existing quantization frameworks is that local reconstruction loss serves as a reliable proxy for global performance degradation. However, we reveal a critical input variance bias in this assumption: for projection layers immediately following activation functions (e.g., $proj\_o$ and $proj\_down$), the input distribution has significantly lower variance due to the squashing effects of activation functions, leading to underestimated Hessian values and artificially low local loss scores. Our controlled experiments show that quantizing these layers causes disproportionate PPL spikes that the local loss fails to predict, which motivates our calibrated sensitivity weighting mechanism.

3.3 Pre-Quantization Profiling via FP16 Forward Simulation

Unlike traditional methods that require re-training or extensive layer-wise optimization, DMQ performs a lightweight profiling step with explicit calibration and loss calculation rules, as detailed below:

Calibration Data: We sample consecutive sequences from the WikiText dataset as calibration data, with a sequence length of 2048 tokens and 64 samples, which is the same as with GPTQ setting. The calibration data is tokenized using the model's native tokenizer and normalized to the FP16 precision.

GPTQ Loss Calculation: We compute the GPTQ loss as the *per-channel mean squared error (MSE)* between the output of the original FP16 operator and the output of the simulated quantized operator. Critically, we do *not* update the original FP16 weights (unlike the original GPTQ[1] which optimizes weights via second-order descent). This process can be efficiently implemented as a single forward pass with simulated quantization, eliminating the need for iterative optimization and significantly reducing profiling time.

Quantization Settings: We fix the quantization group size to 128, a de facto standard in LLM PTQ [1,2]. For a group size of 128 and FP16 scalar and zero point, bpw overhead is calculated as: $\frac{16\ bits/group}{128\ weights/group} = 0.25$.

3.4 Operator Sensitivity Weighting via Architectural Rules

While reconstruction loss provides a baseline, we observe that specific operators in Transformers require explicit sensitivity adjustment. We introduce a rule-based calibration that modifies the sensitivity s_i based on structural properties, rather than relying on global PPL spikes or local loss proxies.

Hessian Diagonal as Base Sensitivity To quantify the global impact of operator-wise quantization error on model performance, we adopt the diagonal of the Hessian matrix as the base sensitivity s_{base} for each linear operator, following the second-order framework of HAWQ [5], with lightweight adaptations for our low-overhead DMQ pipeline.

Formally, for operator o_i, we compute an approximate Hessian H following GPTQ's Cholesky processing pipeline, to stabilize the Hessian estimation. We use only the diagonal elements of H for two core advantages: first, they retain core curvature information as a reliable first-order approximation of global sensitivity; second, they can be efficiently computed during our FP16 forward simulation, with no iterative backpropagation or parameter updates, fully aligning with DMQ's low-overhead design.

We use the *maximum diagonal value* of the processed H to measure quantization sensitivity: a larger value means the operator is more sensitive to quantization noise:

$$s_{base} = \max_{j=1} |H_{j,j}|$$

Selecting the maximum value of the Hessian diagonal is because it can characterize the worst-case quantization error of operators, avoiding high-sensitivity sub-regions being masked by the mean value, which better meets the robust deployment requirements under fixed memory constraints. This base sensitivity bridges the gap between local reconstruction loss and global end-to-end perplexity (PPL) degradation, and serves as the unified foundation for our subsequent architecture-specific sensitivity adjustments.

Down-Projection Compression Ratio Adjustment In Transformer FFN blocks, the down-projection layer (Down-proj) compresses high-dimensional intermediate features back to the model's hidden size, acting as a critical information bottleneck whose output is directly fed to subsequent layers.

This dimensionality reduction makes Down-proj inherently more sensitive to quantization for two key reasons: first, its large compression ratio $r = \frac{intermediate\_size}{hidden\_size}$ (e.g., ~2.7x for Llama-2-7B) amplifies quantization noise via linear aggregation of high-dimensional inputs, leading to error propagation across the model; second, GPTQ propagates quantization errors row-wise[10], and the Down-proj layer has a much larger number of weight matrix rows, magnifying the total quantization error;

third, preceding non-linear activations reduce input variance via their squashing effect, leading to an underestimated base Hessian sensitivity that fails to capture the true global impact of Down-proj quantization.

To compensate for this underestimation and noise amplification, we scale the base sensitivity of Down-proj proportionally to its compression ratio:

$$s_{down-proj} = s_{base} \times \frac{intermediate\_size}{hidden\_size}$$

This adjustment is fully adaptive to different model architectures with no manual hyperparameter tuning. Ablation results confirm it reduces end-to-end PPL by 0.3 - 0.5 under the same memory budget, mitigating accuracy degradation from over-quantization of this bottleneck layer.

V-Proj Numerical Distribution: Natural language has inherently sparse and local semantic dependencies: the semantic meaning of each token only relies on a tiny subset of key context tokens. In natural language understanding and generation tasks, the output of the Softmax function in self-attention modules (i.e., the attention weight $z = Softmax(QK^{\top}/\sqrt{d_k})$) universally exhibits a highly concentrated distribution, a well-validated phenomenon in Transformer-based language models [11,12].

The exponential amplification effect of Softmax further intensifies this sparsity: it suppresses the weights of non-critical tokens to near zero, while concentrating over 80% of the probability mass on only 1-2 dominant tokens, forming a near one-hot distribution in practice. Based on the second-order optimization framework of GPTQ, the amplification factor of a layer's quantization error to the global model loss is proportional to the trace of the input's second moment matrix $tr(H) = \mathbb{E}[\|z\|_2^2]$. For the highly concentrated input of the V-proj layer, $\mathbb{E}[\|z\|_2^2] \approx 0.8 \sim 0.9$, which is roughly twice the value of dense hidden state inputs in other linear projection layers (e.g., Q-proj, K-proj, FFN layers, where $\mathbb{E}[\|x\|_2^2] \approx 0.4 \sim 0.5$). We thus calibrate the sensitivity of V-proj layers with a fixed multiplier $S_v \approx 2$, to protect these error-sensitive layers from aggressive low-bit quantization.

3.5 Dynamic Programming for Optimal Precision Allocation

The optimization problem defined in Equations 1 and 2 is a classic grouped knapsack problem: each operator corresponds to a group, each precision option in the group corresponds to an item with weight $v_{i,b}$ and cost $s_i \cdot e_{i,b}$, we must select exactly one item from each group, the total weight cannot exceed the knapsack capacity V_{max}, and the total cost is minimized.

We solve this problem deterministically using dynamic programming (DP). We define the DP state as:

$dp[i][v]$: the minimum total weighted quantization error when processing the first i operators, using no more than v VRAM.

The state transition equation is:

`

$$dp[i][v] = \min_{b \in \mathcal{B}} \left(dp[i-1][v - v_{i,b}] + s_i \cdot e_{i,b} \right), \quad \forall v \geq v_{i,b} \tag{3}$$

The initial state is:

$$dp[0][0] = 0, \quad dp[0][v] = +\infty, \forall v > 0 \tag{4}$$

After filling the DP table, the minimum total weighted error under the VRAM budget V_{max} is $dp[N][V_{max}]$. We then backtrack the DP table to obtain the optimal precision b_i for each operator.

The time complexity of the DP algorithm is $O(N \cdot M \cdot V_{max})$, where N is the number of operators, M is the number of candidate precisions (fixed to 5 in our work), and V_{max} is the VRAM budget. We solve this via Dynamic Programming to find the globally optimal bit-width assignment in milliseconds.

3.6 Boundary Handling

We define clear boundary conditions to handle extreme VRAM budget scenarios:

1. If the VRAM budget V_{max} is greater than or equal to the total VRAM footprint of the full INT8 model, we assign INT8 precision to all operators, as INT8 provides the best accuracy in our candidate set.

2. If the VRAM budget V_{max} is less than the total VRAM footprint of the full INT3 model, we consider this scenario out of the scope of this work, as even the lowest recommended precision INT3 cannot fit the entire model into the GPU.

4 EXPERIMENTS

4.1 Experimental Setup

Hardware Platforms Our experiments span two types of hardware platforms, corresponding to the *offline quantization phase* and *inference deployment phase*, respectively:

- **Quantization Server**: A workstation equipped with Intel Xeon Gold 6226R CPU @ 2.90GHz, NVIDIA GeForce RTX 3090 (24GB VRAM), CUDA 12.8. All quantization operations (FP16 forward simulation, sensitivity calibration, DP-based precision allocation) are performed offline on this server (*no quantization-related computation is required on edge/embedded devices*), which only load the pre-quantized model weights for inference.
- **Deployment Targets**: We evaluate deployment feasibility on typical resource-constrained devices, including consumer-grade GPUs (NVIDIA T400, RTX 3060), edge computing boards (Raspberry Pi 5), and mobile SoCs (Snapdragon 8 Gen 3). These devices feature fixed, rigid VRAM/RAM budgets, and the core challenge is to *fit the quantized model into the on-device memory* (i.e., solve the "OOM problem"), rather than optimizing inference speed in this work.

Target Models and Datasets Target Models: Qwen2.5-7B, Llama-2-7B, Llama3-8B, Llama3.1-8B, Qwen3-8B, Qwen3-4B (all transformer-based decoder-only LLMs). Evaluation Datasets: WikiText-2 and C4 for perplexity (PPL) evaluation; downstream task evaluation (text generation, commonsense reasoning, QA) will be added in the revised version with complete experimental results.

Table 1: Deployment feasibility of 7B models on constrained devices. × = OOM (Out-of-Memory), √ = Success (model loaded and inferable). Memory values are the total on-device memory footprint (excluding OS overhead for edge/mobile devices).

Device	VRAM/RAM	Uniform 4-bit (GPTQ)	DMQ (Mixed-Precision)
NVIDIA T400	4GB	× (Needs 4.5GB)	√ (3.8GB)
NVIDIA RTX 3060	8GB	√ (Tight, 7.9GB)	√ (Optimized, 6.2GB)
Raspberry Pi 5	8GB	× (OS Overhead, Needs 8.5GB)	√ (5.2GB)
Snapdragon 8 Gen 3	8GB	× (Mobile Memory Limit)	√ (4.9GB)

4.2 Deployment Feasibility on Constrained Devices

A core design goal of DMQ is to ensure that LLMs can be successfully loaded and inferred on resource-constrained devices with fixed memory budgets. Since quantization is an offline process (completed on a high-performance GPU server), embedded/edge devices only need to load the pre-quantized model weights, which eliminates the need for on-device computation during quantization. For the deployment phase, we focus on verifying the memory fitting capability of DMQ (solving the OOM problem), and leave the optimization of inference speed (e.g., kernel fusion, tensor parallelism) for future work.

Table 1 shows the deployment feasibility of 7B-scale LLMs on typical constrained devices, where we compare uniform 4-bit quantization (GPTQ) with DMQ mixed-precision quantization in terms of memory footprint and OOM status. The uniform 4-bit quantization results in a fixed memory footprint that often exceeds the device's VRAM/RAM limit (or runs in a tight state), while DMQ precisely allocates precision to fit the model into the device memory with minimal accuracy loss.

Table 2: Comparison of different quantization search strategies.

Method	**Search Strategy**	**Time Cost**	**Flexibility**
AMQ	Greedy/Heuristic Search	Hours	2,3,4 bit
HAWQ/PTQ4 Bit	Hessian/Second-order	Hours	4,8 bit
DMQ (Ours)	Direct Simulation + DP	Minutes(Profile) + ms(Solve)	3,4,5,6,8 bit

From **Table 1**, we observe that DMQ achieves successful deployment on all tested constrained devices, while uniform 4-bit quantization fails on 3 out of 4 devices due to fixed memory footprint. For the RTX 3060 (8GB), DMQ further optimizes the memory footprint by 21.5% compared to uniform 4-bit quantization, leaving extra memory for KV Cache expansion (supporting longer context lengths). This validates that DMQ effectively solves the "fit or not" core problem for LLM deployment on fixed-memory-constrained devices.

Table 3: Perplexity (PPL) comparison of DMQ against Fixed-Precision and AMQ on various models. And ablation of sensitivity adjustment: 1. without hessian diagonal correction (DMQ w/o1), 2. without downproj/vproj correction (DMQ w/o2).

Llama-3.1-8B				Llama3-8B				Llama2-7b-hf			
Quant	**Size**	**Wiki**	**C4**	**Quant**	**Size**	**Wiki**	**C4**	**Quant**	**Size**	**Wiki**	**C4**
fix-3	2.64	7.53	13.34	fix-3	2.64	7.65	14.76	fix-3	2.45	6.26	8.51
fix-4	3.45	6.44	10.22	fix-4	3.45	6.35	10.15	fix-4	3.20	5.65	7.32
fix-5	4.27	6.26	9.54	fix-5	4.27	6.14	9.44	fix-5	3.96	5.57	7.17
fix-6	5.08	6.21	9.40	fix-6	5.08	6.08	9.32	fix-6	4.71	5.54	7.14
AMQ	2.73	8.24	14.45	AMQ	2.75	7.22	12.54	AMQ	2.50	6.08	8.28
	2.97	7.78	13.38		3.00	6.84	11.46		2.75	5.88	7.81
	3.22	7.55	12.71		3.22	6.59	10.81		2.99	5.73	7.52
DMQ (w/o1)	3.00	7.01	11.83	DMQ (w/o1)	3.00	7.05	12.41	DMQ (w/o1)	3.00	5.80	7.58
DMQ (w/o2)	3.00	7.04	11.92	DMQ (w/o2)	3.00	7.15	12.41	DMQ (w/o2)	3.00	5.84	7.64
DMQ	2.65	7.51	13.26	DMQ	2.65	7.60	13.84	DMQ	2.46	6.25	8.49
	3.00	6.99	11.91		3.00	7.03	12.27		3.00	5.78	7.59
	3.46	6.50	10.39		3.46	6.46	10.43		3.50	5.61	7.25
	4.00	6.31	9.78		4.00	6.22	9.72		4.50	5.55	7.14
	4.50	6.24	9.54		4.50	6.14	9.45		4.71	5.54	7.14
	5.08	6.21	9.40		5.08	6.08	9.32				
Qwen3-8B				**Qwen2.5-7B-Instruct**				**Qwen3-4B**			
Quant	**Size**	**Wiki**	**C4**	**Quant**	**Size**	**Wiki**	**C4**	**Quant**	**Size**	**Wiki**	**C4**
fix-3	2.63	10.31	16.93	fix-3	2.47	8.17	14.26	fix-3	1.37	14.33	22.07
fix-4	3.44	9.66	15.43	fix-4	3.23	7.41	12.45	fix-4	1.80	13.10	20.06
fix-5	4.25	9.40	15.05	fix-5	3.99	7.28	12.17	fix-5	2.22	13.21	19.56
fix-6	5.05	9.35	14.96	fix-6	4.75	7.24	12.06	fix-6	2.64	12.99	19.37
AMQ	2.74	10.23	16.79	AMQ	2.50	8.12	14.07	AMQ	1.52	14.50	21.76
	2.99	9.98	16.07		3.00	7.48	12.93		1.76	13.43	20.24
	3.24	9.82	15.69								
DMQ (w/o1)	3.00	9.80	15.95	DMQ (w/o1)	3.00	7.56	12.78	DMQ (w/o1)	1.50	13.56	20.88
DMQ (w/o2)	3.00	9.89	16.16	DMQ (w/o2)	3.00	7.55	12.79	DMQ (w/o2)	1.50	13.89	20.92
DMQ	2.63	10.30	16.93	DMQ	2.47	8.16	14.24	DMQ	1.50	13.28	20.86
	3.00	9.81	15.89		3.00	7.51	12.70		1.80	13.29	20.07
	3.44	9.60	15.35		3.25	7.39	12.41		2.00	13.08	19.81
	4.00	9.42	15.09		3.99	7.27	12.18		2.23	13.10	19.53
	4.25	9.36	14.97		4.50	7.24	12.08		2.50	12.97	19.33
	5.05	9.35	14.96		4.78	7.24	12.06		2.64	12.99	19.37

4.3 Quantization Process Efficiency and Accuracy

We compare DMQ against the state-of-the-art AutoML-based mixed-precision method, AMQ[7], which uses evolutionary algorithms to search for Pareto-optimal solutions. Furthermore, since AMQ cannot fit the model exactly into the memory budget, we need to manually adjust the search space to find a quantization scheme.
We evaluate both AMQ and DMQ and get the best quantization scheme under the varying memory budget, perform inference and compare their perplexity (PPL) with DMQ framework.

Time and Memory Overhead Comparison **Table 2** and **Table 4** detail the search strategy and the wall-clock time and peak VRAM usage for the quantization process. DMQ eliminates the need for stochastic search, resulting in a massive reduction in time and resource consumption.

Table 4: Quantization Time and Peak VRAM Comparison (Llama2-7B).

Stage	AMQ Time	AMQ VRAM	DMQ Time	DMQ VRAM
Pre-quantization	235.73s	4.5GB	1495.36s	12.77GB
Sensitivity/Proxy	325.33s	20.18GB	-	-
Search(Evolutionary)	20515.97s	20.18GB	-	-
Final Selection/Apply	0.96s	CPU Only	1.03s	CPU Only
Total	**21077.99s(5.8h)**	**20.18GB**	**1496.39s(25m)**	**12.77GB**

Accuracy Comparison **Table 4** and Fig.**1** demonstrate: Fixed-precision methods suffer from memory-accuracy staircase effects (e.g., Llama-3.1-8B INT3 vs. INT4 creates a 0.81GB dead zone). AMQ underperforms DMQ by 0.2~0.5 PPL on WikiText-2 due to local optima and proxy errors. DMQ fills dead zones, outperforming both baselines at all budgets—e.g., 3.00GB Llama-3.1-8B achieves 6.99 WikiText-2 PPL (vs. AMQ's 7.78 and INT3's 7.53)—and maintains consistency across 4B~8B models.

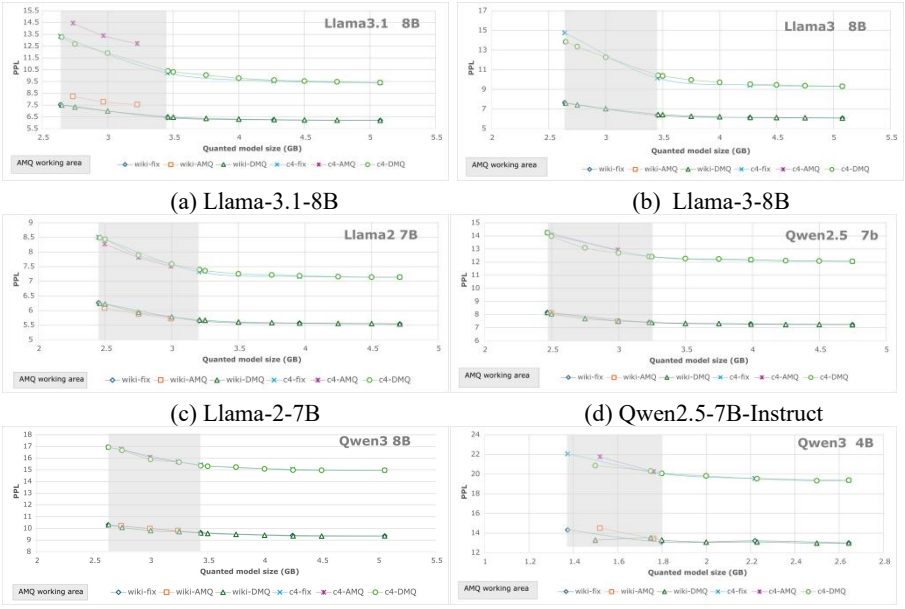
Analysis:

- *Speedup:* DMQ reduces the total processing time from approximately 5.8 hours to under 25 minutes. The core difference lies in the search phase: AMQ requires over 5 hours of evolutionary search, whereas DMQ solves the allocation via Dynamic Programming in milliseconds (sub-second).
- *Determinism:* Unlike AMQ, which may yield different results on different runs, DMQ provides a globally optimal and deterministic solution.
- *Accuracy:* While AMQ relies on proxy models (HQQ) which introduce approximation errors, DMQ uses direct GPTQ error metrics calibrated with sensitivity, leading to more accurate allocation decisions for more models.

Ablation of Sensitivity Adjustments To validate the effectiveness of our three-stage sensitivity weighting mechanism (Hessian diagonal base sensitivity, Down-proj

compression ratio adjustment, V-proj numerical distribution calibration), we conduct ablation experiments on all tested models, with two key ablation variants: DMQ w/o1: Removing the Hessian diagonal correction (using raw local reconstruction loss as the base sensitivity instead of the GPTQ-based approximate Hessian diagonal maximum). DMQ w/o2: Retaining the Hessian diagonal base sensitivity but removing the architecture-specific calibration for Down-proj and V-proj layers.

Fig. 1: Perplexity (PPL) comparison of DMQ



(a) Llama-3.1-8B (b) Llama-3-8B

(c) Llama-2-7B (d) Qwen2.5-7B-Instruct

(e) Qwen3-8B (f) Qwen3-4B

Table 5 shows the downstream task performance of AMQ and DMQ on Qwen3-4B and Qwen3-8B under tight memory constraints. DMQ consistently achieves lower perplexity on both WikiText-2 and C4 datasets while maintaining comparable or higher accuracy across all downstream benchmarks, including acc challenge(M-C), acc easy(M-E), PIQA, Winogrande, SciQ, MNLI, and HellaSwag. Notably, under the same memory budget, DMQ reduces the Wiki PPL from 14.30 to 13.28 for Qwen3-4B and from 9.98 to 9.81 for Qwen3-8B, demonstrating its ability to maintain better language modeling quality and generalization ability in low-memory environments.

4.4 Future Work

We will focus on three research directions: 1), explore the feasibility of lower-bit mixed-precision quantization (e.g., 2-bit). For Llama3.1-8B, uniform 2-bit quantization leads to severe accuracy degradation (1.8281GB, Wiki PPL 720.9476, C4 PPL 2415.2324), yet empirical results show most operators can be quantized to 2-bit with only a few critical ones retained in higher bits—we will optimize the Pareto-optimal boundary to mitigate the downstream task invalidation caused by 2-bit near-

optimal schemes. 2), adapt the framework to multimodal models and MoE architectures, designing dynamically adjustable calibration coefficients for complex model structures to flexibly meet mixed-precision requirements on low-end devices. 3), extend the method to support mixed-precision quantization of KV Cache along the long-context sequence dimension, further optimizing memory usage for long-sequence inference scenarios.

Table 5: Downstream task performance on Qwen3 under tight memory budgets.

Model	Method	Quant (GB)	Wiki	C4	M-C	M-E	PIQA	Winog-rande	SciQ	MNLI	Hella-Swag
Qwen3	AMQ	1.50	14.30	21.92	0.55	0.82	0.75	0.64	0.97	0.63	0.65
-4B	DMQ	1.50	13.28	20.86	0.53	0.82	0.75	0.64	0.96	0.65	0.63
Qwen3	AMQ	2.99	9.98	16.07	0.63	0.86	0.78	0.68	0.97	0.73	0.74
-8B	DMQ	3.00	9.81	15.89	0.62	0.86	0.79	0.69	0.98	0.75	0.73

References

[1] E. Frantar and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers," arXiv preprint arXiv:2210.17323, 2022.

[2] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, "Awq: Activation-aware weight quantization for llm compression and acceleration," arXiv preprint arXiv:2306.00978, 2023.

[3] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, and I. Stoica, "vllm: A high-throughput and memory-efficient inference and serving system for large language models," arXiv preprint arXiv:2309.06180, 2023.

[4] S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "Smoothquant: Accurate and efficient post-training quantization for large language models," arXiv preprint arXiv:2211.10438, 2022.

[5] Z. Yao, Z. Dong, Z. Zheng, A. Gholami, J. Yu, E. Tan, L. Wang, Q. Huang, Y. Wang, M. W. Mahoney, and K. Keutzer, "Hawqv3: Dyadic neural network quantization," in International Conference on Machine Learning (ICML). PMLR, 2021, pp. 10764–10774.

[6] H. F. P. Team, "Ptq4bit: Post-training quantization for 4-bit llms," GitHub repository: https://github.com/huggingface/peft, 2023.

[7] Y. Li, X. Liu, and J. Han, "Amq: Auto mixed-precision quantization for llms," arXiv preprint arXiv:2402.09353, 2024.

[8] A. Zhou, S. Zhou, Y. Wu, X. Zhou, C. Leng, and J. Cheng, "Mixed precision quantization for deep neural networks," in AAAI Conference on Artificial Intelligence (AAAI), vol. 33, no. 01, 2019, pp. 8252–8259.

[9] Y. Li, G. Wang, B. Ni, W. Zhang, and X. Yang, "Automatic channel pruning via feature map reconstruction," in IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020, pp. 12 486–12 495.

[10] H. Duanmu, X. Li, Z. Yuan, S. Zheng, J. Duan, X. Zhang, and D. Lin, "Mxmoe: Mixedprecision quantization for moe with accuracy and performance co-design," in International Conference on Machine Learning, 2025.

[11] P. Michel, O. Levy, and G. Neubig, "Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned," in Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL), 2019, pp. 5797–5808.

[12] S. Serrano and N. A. Smith, "Is attention interpretable?" in Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL), 2019, pp. 2931–2951.